

Nepal Banking Risk Analytics Dashboard Documentation

<https://nepal-banking-risk-analytics.streamlit.app/>

Nishanta Nepal

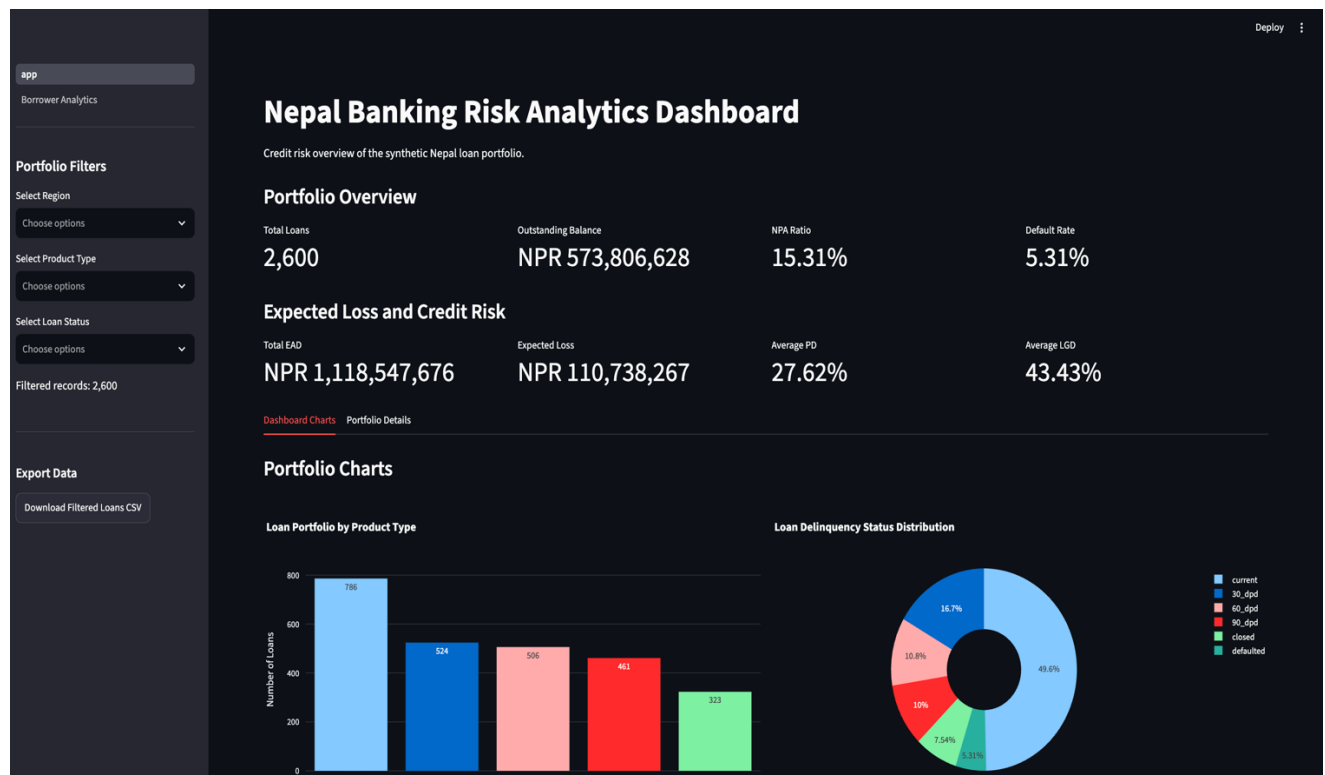
Master's in computer science

Technische Universität Dresden

Dresden, Germany

nepalnishanta29@gmail.com

Nepal Banking Risk Analytics Dashboard is an interactive Streamlit application for monitoring loan portfolio health, credit risk KPIs, and borrower trends using synthetic Nepal-style banking data. It helps analyze NPA, default risk, expected loss, portfolio composition, and borrower segments through intuitive dashboards.



Overview

Nepal Banking Risk Analytics Dashboard is a portfolio project built as a Streamlit web application for exploring credit risk metrics on a synthetic Nepal-style banking dataset. The application focuses on portfolio-level monitoring and borrower-level analysis using interactive charts, KPI cards, filters, and downloadable data exports.

The project was developed in a step-by-step way, starting from project setup, dataset creation, modular data loading, KPI calculations, charts, filters, risk visualizations, export functionality, and multipage borrower analytics.

Project goal

The main goal of the project is to simulate how a banking or lending analytics team might monitor loan portfolio health using metrics such as NPA ratio, default rate, probability of default, loss given default, exposure at default, and expected loss.

A second goal is to present the project as a strong GitHub portfolio piece with a practical software structure rather than a single notebook or a one-file dashboard.

Dataset used

The project uses a synthetic dataset created specifically for this dashboard project. It includes four CSV files:

- nepal_synthetic_bank_loans.csv
- nepal_synthetic_borrowers.csv
- nepal_synthetic_bank_dataset.csv
- dataset_summary.csv

The dataset was designed to support portfolio risk analysis safely without relying on private bank records, which is appropriate for public portfolio projects and GitHub publication.

Main loan fields

The loan-level data includes fields such as:

- loan_id
- borrower_id
- origination_date
- maturity_date
- product_type
- region

- status
- loan_amount_npr
- current_balance_npr
- ead
- pd
- lgd
- expected_loss_npr
- defaulted_flag
- npa_flag
- vintage

These fields allow the dashboard to calculate credit risk KPIs and create time-based, segment-based, and portfolio risk visualizations.

Main borrower fields

The borrower-level data includes fields such as:

- borrower_id
- age
- gender
- marital_status
- education_level
- employment_type
- region
- sector
- monthly_income_npr
- credit_score
- existing_delinquency_flag

These fields support customer segmentation and borrower analytics views.

Application architecture

The project follows a modular Streamlit structure. Streamlit supports multipage applications through the `pages/` directory, which is why the project separates the main dashboard from the borrower analytics page.

File or folder	Purpose
app.py	Main portfolio dashboard page
pages/1_Borrower_Analytics.py	Borrower analytics multipage view

File or folder	Purpose
datasets/	CSV input data
src/data_loader.py	CSV loading functions
src/kpi_calculations.py	Loan portfolio KPI logic
src/charts.py	Plotly chart functions
src/filters.py	Portfolio filter logic
src/borrower_analysis.py	Borrower KPI logic
src/borrower_filters.py	Borrower page filter logic
src/export_utils.py	CSV export helper
src/portfolio_analysis.py	Product risk summary logic

This modular design improves readability, makes testing easier, and avoids placing all analytics logic in one file.

What was built

The project was developed in stages.

1. Initial setup

The first step was creating a GitHub project, local project folder, Python virtual environment, requirements file, and a minimal Streamlit app that successfully opened in the browser.

2. Data loading layer

A dedicated loader module was created to read the CSV files using Pandas. `pandas.read_csv()` is the standard CSV loading function for this workflow, and the project uses it to load loan, borrower, combined, and summary data into DataFrames.

3. KPI calculations

A separate KPI calculation module was added to compute portfolio-level metrics such as:

- total loans
- outstanding balance
- total EAD
- NPA ratio
- default rate
- expected loss
- average PD
- average LGD

These KPIs form the top summary cards in the dashboard.

4. Portfolio charts

The next stage added Plotly-based portfolio charts rendered with Streamlit. Streamlit displays Plotly figures with `st.plotly_chart()`, which supports interactive data visualization inside the app.

The first portfolio charts included:

- loan product mix
- delinquency status distribution
- NPA ratio by region
- loan origination trend
- PD vs LGD risk scatter
- vintage analysis by origination cohort

Plotly Express supports bar, pie, line, and scatter-based charts, which makes it well suited for this dashboard workflow.

5. Sidebar portfolio filters

Sidebar filters were then added for region, product type, and loan status using Streamlit multiselect widgets. `st.multiselect()` allows the user to select multiple category values and filter the DataFrame interactively.

This made the dashboard dynamic rather than static, because KPI cards and charts update from the filtered portfolio slice.

6. Product summary and tabs

The portfolio page was later improved using tabs and grouped summary tables. Streamlit tabs provide a cleaner way to organize more content on the same page without making the screen too crowded.

Grouped product-level summaries were created using Pandas group-by aggregation, which is the standard approach for segmented summary tables.

7. Data export

A filtered CSV export function was added so users can download the currently filtered loan slice. Streamlit supports this pattern through `st.download_button`, which is commonly used to export generated CSV data from a DataFrame-driven app.

8. Borrower analytics page

A second page was added for borrower analytics using Streamlit's `multiselect` `pages/` directory. This page focuses on borrower-level KPIs and segmentation charts rather than portfolio-level loan risk views

The borrower page includes:

- total borrowers
- average monthly income
- average credit score
- existing delinquency rate
- income distribution chart
- credit score distribution chart
- borrowers by employment type
- borrowers by region

9. Borrower page filters

Borrower-specific filters were added for region, employment type, education level, and existing delinquency flag using the same sidebar multiselect pattern used in the main dashboard.[docs.streamlit+1](#)

Main metrics explained

NPA ratio

NPA ratio measures the share of loans that are considered non-performing. In this project, a loan is flagged as NPA when it is 90 or more days past due or already defaulted.

PD

Probability of Default estimates how likely a borrower or loan is to default.

LGD

Loss Given Default estimates the share of exposure likely to be lost if the borrower defaults.

EAD

Exposure at Default measures the amount exposed at the time of default.

Expected loss

Expected loss combines PD, LGD, and EAD into one risk estimate:

$$\textbf{Expected Loss} = \textbf{PD} \times \textbf{LGD} \times \textbf{EAD}$$

This metric is important because it gives a direct estimate of portfolio credit risk in monetary terms.

Charts explained

Product mix chart

Shows the number of loans by product type, helping users understand how the portfolio is distributed.

Delinquency distribution chart

Shows the share of loans across delinquency states such as current, 30 DPD, 60 DPD, 90 DPD, closed, and defaulted.

NPA by region chart

Shows which regions have relatively higher non-performing loan rates.

Origination trend chart

Shows how loan origination volume changes over time.

PD vs LGD scatter chart

Shows the relationship between default probability and loss severity, while bubble size adds EAD as a third dimension.

Vintage analysis chart

Shows how cohorts of loans grouped by origination period differ in NPA ratio.

Why the project is useful

This project is useful as a portfolio piece because it combines data engineering, risk analytics, dashboard building, filtering, modular Python code organization, and user-facing export functionality in one application.

It also reflects a realistic analytics workflow: load data, clean and structure it, calculate KPIs, visualize portfolio trends, filter segments, and export results for further review.

Possible next improvements

Some meaningful next improvements are:

- add borrower-loan join analysis across pages
- add more advanced vintage and roll-rate analysis
- add model-based risk scoring or default prediction
- add better styling and UI polish
- deploy the app publicly using Streamlit Community Cloud
- add tests for KPI and filter functions

Conclusion

Nepal Banking Risk Analytics Dashboard is a well-structured analytics project built around synthetic Nepal-style banking data. It demonstrates practical use of Python, Pandas, Plotly, Streamlit, modular project design, and credit risk concepts in a way that is suitable for both learning and portfolio presentation.